

Лабораторная работа 1

Исследование процессов обнаружения, сопряжения и установления соединения в сетях Bluetooth

1.1 Цель работы

Получение базовых теоретических знаний о механизмах работы сетей Bluetooth, а также практических навыков управления процессами поиска (inquiry), сопряжения (pairing) и установления соединения (connection) с использованием официального стека BlueZ в ОС семейства Linux.

1.2 Задачи

- Изучить теоретические основы процедур Inquiry, Page, Pairing и Connection в архитектуре Bluetooth BR/EDR.
- Ознакомиться с утилитами управления стеком BlueZ и анализатором трафика btmon.
- На практике выполнить сканирование окружающих Bluetooth-устройств, зафиксировать их адреса и классы устройств.
- Выполнить процедуру сопряжения с заданным устройством.
- Установить активное логическое соединение и проверить его состояние.
- Протоколировать этапы работы для оформления отчета.

1.3 Теоретический материал

1.3.1 Процесс обнаружения устройств (Inquiry и Page)

В сетях Bluetooth обнаружение устройств состоит из двух последовательных фаз: **Inquiry** (поиск) и **Page** (вызов).

- **Inquiry scan**: Устройства, находящиеся в режиме обнаружения, периодически прослушивают эфир на специальных частотах (Inquiry hopping sequence).
- **Inquiry**: Иницилирующее устройство (Inquirer) рассылает ID-пакеты по частотной сетке поиска. Устройства в режиме сканирования отвечают пакетом **FHS (Frequency Hop Synchronization)**, который содержит 48-битный адрес устройства (BD_ADDR), тип часов, класс устройства (CoD) и параметры синхронизации.

После получения FHS-пакетов инициатор формирует список найденных устройств. Для перехода к следующему этапу используется процедура **Page**, в ходе которой устройства синхронизируются по часам и устанавливаются канал управления (LMP — Link Manager Protocol).

1.3.2 Процесс сопряжения (Pairing и Bonding)

Сопряжение — это процесс безопасного обмена ключами между двумя устройствами для установления защищённого канала.

- **Pairing** (сопряжение): процедура обмена аутентификационными ключами. В современных спецификациях (версия 2.1+) используется механизм **SSP (Secure Simple Pairing)**, основанный на алгоритме Диффи-Хеллмана (ECDH). SSP поддерживает несколько моделей ассоциации: Numeric Comparison, Just Works, Passkey Entry и Out of Band.
- **Bonding** (связывание): сохранение сгенерированного **Link Key** в энергонезависимой памяти устройств. После связывания последующие соединения могут устанавливаться автоматически без повторного прохождения процедуры аутентификации.

В случае успешного сопряжения активируется шифрование на уровне контроллера (Baseband), обеспечивающее конфиденциальность передаваемых данных.

1.3.3 Установление соединения (Connection)

После успешного Page/Pairing на уровне хоста (Host) создаётся логический канал через подуровень **L2CAP (Logical Link Control and Adaptation Protocol)**. L2CAP обеспечивает мультиплексирование нескольких протоколов более высокого уровня над одним физическим каналом. Для установления соединения используются профили (например, **RFCOMM** для эмуляции последовательного порта, **A2DP** для аудио, **HID** для ввода). Клиент запрашивает соединение у серверного процесса, указывая PSM (Protocol/Service Multiplexer). После установки L2CAP-соединения устройства могут обмениваться полезными данными в рамках выбранного профиля.

1.3.4 Стек BlueZ и основные инструменты управления

BlueZ — официальный стек протоколов Bluetooth для ядра Linux. Основной пользовательский интерфейс взаимодействия со стеком предоставляется через интерактивную утилиту `bluetoothctl`. Управление осуществляется через D-Bus API демона `bluetoothd`.

Таблица 1.1: Основные команды утилиты `bluetoothctl`

Команда	Описание
<code>power on</code>	Включение адаптера Bluetooth
<code>scan on</code>	Запуск режима сканирования (Inquiry)
<code>scan off</code>	Остановка сканирования
<code>devices</code>	Вывод списка обнаруженных или сопряжённых устройств
<code>pair <BD_ADDR></code>	Запуск процедуры сопряжения с указанным устройством
<code>trust <BD_ADDR></code>	Добавление устройства в список доверенных (автосопряжение)
<code>connect <BD_ADDR></code>	Установление логического соединения
<code>info <BD_ADDR></code>	Вывод подробной информации об устройстве (ключи, профили, статус)
<code>quit / exit</code>	Завершение сеанса работы с утилитой

Для мониторинга низкоуровневого трафика (HCI-команды, события, ACL-пакеты) в пакете BlueZ предусмотрен анализатор `btmon`, аналогичный по назначению утилите `tcpdump` в IP-сетях.

1.4 Порядок выполнения лабораторной работы

Примечание. Для выполнения работы требуется наличие активного Bluetooth-адаптера на лабораторном ПК. Все команды, кроме `info` и `devices`, требуют прав суперпользователя или работы в группе `bluetooth`.

1. Запустите терминал и откройте утилиту управления стеком:

```
user@host:~$ sudo bluetoothctl
```

В результате откроется интерактивная оболочка с приглашением `[bluetooth]#`.

2. Проверьте состояние адаптера и включите его при необходимости:

```
[bluetooth]# show
[bluetooth]# power on
```

- Зафиксируйте вывод команды **show** (адрес адаптера, имя, текущее состояние).
3. Запустите процесс сканирования устройств (Inquiry):

```
[bluetooth]# scan on
```

Подождите 10–15 секунд. В терминале будут появляться сообщения вида **Device XX:XX:XX:XX:XX:XX**. После получения достаточного количества устройств выполните:

```
[bluetooth]# scan off
```

4. Выберите одно из обнаруженных устройств для дальнейшей работы. Выведите список найденных устройств и запомните его **BD_ADDR**:

```
[bluetooth]# devices
```

5. Запустите параллельный терминал и начните захват низкоуровневого трафика с помощью **btmon**:

```
user@host:~$ sudo btmon -w bt_trace.log
```

Утилита будет записывать сырые HCI-события в файл **bt_trace.log** в бинарном формате.

6. В первом терминале иницилируйте процедуру сопряжения (Pairing):

```
[bluetooth]# pair <BD_ADDR>
```

Следуйте инструкциям утилиты (при необходимости подтвердите PIN-код или код числового сравнения на обоих устройствах). Дождитесь сообщения **Pairing successful**.

7. Установите логическое соединение (Connection):

```
[bluetooth]# connect <BD_ADDR>
```

Убедитесь в выводе **Connection successful**. Выполните команду **info <BD_ADDR>** и зафиксируйте статус соединения, список доступных **UUID** и состояние шифрования.

8. Завершите работу с **bluetoothctl** и остановите **btmon** (нажатие **Ctrl-C**). Просмотрите захваченный журнал:

```
user@host:~$ btmon -r bt_trace.log
```

Найдите в логе этапы: **Inquiry**, **Create Connection (Page)**, **User Confirmation Request / Link Key**, **Encryption Change**.

9. Выведите утилиту из интерактивного режима:

```
[bluetooth]# disconnect <BD_ADDR>
[bluetooth]# quit
```

1.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Задание согласно варианту.
4. Листинги результатов работы:
 - а) вывод команды **show** до и после включения адаптера;
 - б) список обнаруженных устройств (**devices**) с указанием **BD_ADDR** и класса устройства;
 - в) вывод команд **pair**, **connect**, **info** с комментариями к каждому этапу;
 - г) фрагменты лога **btmon**, иллюстрирующие пакеты **Inquiry**, **Page** и установку шифрования;

- е) скриншоты или текстовые подтверждения статуса соединения.
5. Ответы на контрольные вопросы.
 6. Краткие выводы по работе.

Примечание. Перед каждым листингом должна быть указана выполненная команда. Вывод `btmon` рекомендуется прилагать отдельным файлом или в виде вырезок, подтверждающих прохождение ключевых фаз протокола.

Контрольные вопросы

1. Опишите различие между фазами Inquiry и Page. Какие пакеты используются на каждом этапе?
2. Что такое FHS-пакет и какую информацию он передаёт инициатору поиска?
3. В чём заключается разница между процессами Pairing и Bonding в спецификации Bluetooth?
4. Какие модели ассоциации предусмотрены в механизме Secure Simple Pairing (SSP) и в каких сценариях они применяются?
5. Какую роль играет подуровень L2CAP при установлении соединения между хостами?
6. Как с помощью утилиты `btmon` отфильтровать только события, связанные с шифрованием канала?
7. Почему для работы с адаптером в BlueZ часто требуются права суперпользователя или добавление пользователя в группу `bluetooth`?

Лабораторная работа 2

Исследование архитектуры пикосети Bluetooth и организация обмена данными через RFCOMM

2.1 Цель работы

Изучить принципы построения Bluetooth-пикосетей, технологию скачкообразной перестройки рабочей частоты (FHSS) и механизмы установления логических соединений. Получить практические навыки программирования сетевого взаимодействия в архитектуре «мастер-ведомые» с использованием Python, стека BlueZ и среды разработки Arduino IDE.

2.2 Задачи

- Изучить топологию пикосети, роли устройств (мастер/слейв) и ограничения архитектуры.
- Рассмотреть принцип работы FHSS, структуру частотной сетки и синхронизацию каналов.
- Ознакомиться с протоколом RFCOMM, его назначением и особенностями эмуляции последовательного порта.
- Освоить процесс подготовки среды Arduino IDE для работы с ESP32 и загрузки прошивки.
- Модифицировать базовую прошивку узла.
- Настроить два модуля ESP32 в режиме Bluetooth Classic SPP-серверов.
- Разработать и запустить Python-скрипт на ПК, устанавливающий параллельные RFCOMM-соединения с обоими устройствами.
- Организовать двунаправленный обмен данными, проверить наличие ФИО в возвращаемых пакетах.

2.3 Теоретический материал

2.3.1 Архитектура пикосети и роли устройств

Базовой единицей любой сети Bluetooth является **пикосеть** (piconet). Она представляет собой логическую сеть, образованную одним ведущим устройством (**мастером**) и до семи активных ведомых устройств (**слейвов**). Дополнительно в пикосети могут находиться до 255 устройств в режиме ожидания (паркованные устройства, parked), не участвующие в активном обмене данными, но готовых к активации.

Мастер выполняет функции синхронизации: его внутренние часы и 48-битный Bluetooth-адрес (BD_ADDR) используются для генерации псевдослучайной частотной последовательности и определения временных слотов. Все слейвы синхронизируются по часам мастера и следуют единому расписанию скачков частоты. В рамках одной пикосети обмен данными возможен только между мастером и слейвами; прямое взаимодействие слейв-слейв не поддерживается на физическом уровне.

2.3.2 Технология FHSS и организация физического канала

Для обеспечения помехоустойчивости и безопасности в сетях Bluetooth BR/EDR применяется технология **FHSS (Frequency Hopping Spread Spectrum)**. Рабочий диапазон 2,4–2,4835 ГГц делится на 79 каналов с шагом 1 МГц (в некоторых регионах — 23 или 25 каналов). Скорость скачков составляет 1600 переходов в секунду (в базовом режиме BR).

Последовательность каналов вычисляется по алгоритму, использующему 28-битное значение адреса мастера, 28-битные внутренние часы и 1-битный идентификатор режима. Таким образом, каждое устройство в пикосети «знает», на какой частоте будет находиться мастер в следующий временной слот. Параметры физического канала представлены в табл. 2.1.

Таблица 2.1: Параметры физического канала и технологии FHSS

Параметр	Значение / Описание
Частотный диапазон	2400–2483,5 МГц (ISM-диапазон)
Количество каналов	79 (шаг 1 МГц)
Скорость скачков	1600 гоп/с (1,6 кГц)
Время удержания частоты	625 мкс (1 слот)
Синхронизация	По часам мастера и его BD_ADDR
Модуляция	GFSK (Gaussian Frequency-Shift Keying)

Организация канала на уровне контроллера включает: *физический канал* (определяется частотной сеткой), *логический транспортный канал* (асинхронный ACL или синхронный SCO), *логический канал* (для конкретного соединения) и *логический транспортный канал* (каналы данных и сигнализации). Мастер управляет распределением временных слотов между слейвами, используя пакетную передачу данных.

2.3.3 Протокол RFCOMM и программная реализация

Для передачи потоковых данных в архитектуре Bluetooth используется подуровень **RFCOMM**, эмулирующий последовательный порт (эмуляция RS-232). RFCOMM работает поверх L2CAP и использует концепцию *Data Link Connection Identifiers* (DLCI) для мультиплексирования до 60 виртуальных последовательных соединений в рамках одного физического канала.

В ОС семейства Linux реализация RFCOMM доступна через системные сокеты. В языке Python для работы с Bluetooth Classic и установления RFCOMM-соединений используется стандартная библиотека `socket` с доменом `AF_BLUETOOTH` и протоколом `BTPROTO_RFCOMM`. После установления соединения программный интерфейс полностью идентичен работе с обычными TCP-сокетами, что упрощает разработку клиент-серверных приложений для обмена текстовыми и бинарными данными.

2.3.4 Среда разработки Arduino IDE и загрузка прошивки в ESP32

Для программирования микроконтроллеров ESP32 в данной лабораторной работе используется среда **Arduino IDE**. Это кроссплатформенное приложение, позволяющее писать код на C/C++ и загружать его в микроконтроллер через USB-интерфейс.

Основные этапы подготовки и загрузки:

1. **Установка поддержки ESP32:** В настройках Arduino IDE (File → Preferences) в поле *Additional Boards Manager URLs* необходимо добавить ссылку на репозиторий ESP32: <https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package>. Затем в Board Manager устанавливается пакет *esp32 by Espressif Systems*.
2. **Выбор платы и порта:** В меню Tools → Board выбирается конкретная модель (например, *ESP32 Dev Module*). В меню Tools → Port выбирается соответствующий USB-порт (в Linux обычно `/dev/ttyUSB0` или `/dev/ttyACM0`).
3. **Права доступа в Linux:** Для загрузки прошивки пользователь должен иметь права на запись в USB-порт. Обычно это решается добавлением пользователя в группу `dialout`:

```
user@host:~$ sudo usermod -a -G dialout $USER
```

После выполнения команды требуется перезагрузка системы или выход из сеанса.

4. **Компиляция и загрузка:** Код компилируется и загружается нажатием кнопки *Upload* (стрелка вправо). При загрузке модуль ESP32 автоматически переходит в режим программирования (благодаря схеме автопереключения на базе транзисторов или ручному нажатию кнопки BOOT).

2.4 Порядок выполнения лабораторной работы

Примечание. Для выполнения работы требуется ПК с установленным Linux и стекком BlueZ, а также два микроконтроллера ESP32 с поддержкой Bluetooth Classic. Перед началом работы устройства должны быть сопряжены с ПК (см. Лабораторную работу 1).

1. Модификация прошивки (идентификация студента):

- Откройте предоставленный базовый скетч для ESP32 в Arduino IDE.
- Найдите в коде функцию обработки входящих сообщений (блок `if (SerialBT.available())`).
- В месте формирования ответного сообщения найдите переменную, отвечающую за статус или текст ответа (например, `String response = "YOUR MESSAGE";` или аналогичную константу).
- Замените стандартное значение этой переменной на ваше **Фамилию Имя Отчество**

2. **Загрузка прошивки:** Подключите первый модуль ESP32 к ПК, выберите правильный порт и плату в Arduino IDE, затем загрузите модифицированную прошивку. Повторите процедуру для второго модуля. Запишите 48-битные MAC-адреса обоих устройств.

3. **Проверка и освобождение каналов связи:**

- Linux (BlueZ) может автоматически устанавливать соединение с ранее сопряженными устройствами при их включении. Это заблокирует порт для вашего Python-скрипта.

- Проверьте статус подключений командой:

```
user@host:~$ bluetoothctl devices connected
```

- Если ваши устройства (ESP32-Node1/Node2) отображаются в списке подключенных, необходимо принудительно разорвать соединение для каждого из них:

```
user@host:~$ bluetoothctl disconnect XX:XX:XX:XX:XX:XX
```

- Убедитесь, что устройства остаются в состоянии **paired** (сопряжены), но **not connected** (не подключены).

4. Установите необходимые системные пакеты на лабораторном ПК (если не установлены):

```
user@host:~$ sudo apt update
user@host:~$ sudo apt install python3 python3-pip libbluetooth-dev
user@host:~$ pip3 install --user pybluez
```

5. Создайте файл `bt_master.py` и реализуйте скрипт, устанавливающий два параллельных RFCOMM-соединения. Скрипт должен:

- Подключаться к обоим устройствам.
- Отправлять тестовые команды (например, "PING").
- Принимать ответы и проверять, содержится ли в них ваше ФИО.

Базовая структура приведена в листинге 1. Замените `MAC_ESP1` и `MAC_ESP2` на реальные адреса ваших модулей.

6. Запустите скрипт с правами суперпользователя (для доступа к HCI-интерфейсу):

```
user@host:~$ sudo python3 bt_master.py
```

7. Наблюдайте за выводом терминала. Скрипт должен подключиться к устройствам, отправить команду и получить ответ вида `[ACK-NODE2] Received: PING | Status: PONG`.

8. В параллельном окне терминала запустите мониторинг трафика:

```
user@host:~$ sudo btmon -w lab02_trace.log
```

После завершения обмена остановите `btmon` и просмотрите лог, обращая внимание на этапы `L2CAP Connect Request`, `RFCOMM SABM` и `Encryption Change`.

9. Зафиксируйте в отчёте успешные подключения, факт возврата правильного ФИО от обоих устройств, переданные/принятые байты.

Листинг 1: Базовая структура Python-скрипта для управления пикосетью

```
import socket
import time
import bluetooth

# MAC addresses of slave devices (replace with real ones)
```

```

ESP1_MAC = "AA:BB:CC:DD:EE:01"
ESP2_MAC = "AA:BB:CC:DD:EE:02"
def connect_rfcomm(mac_addr, port=RFCOMM_PORT):
    sock = socket.socket(socket.AF_BLUETOOTH,
                          socket.SOCK_STREAM,
                          socket.BTPROTO_RFCOMM)

    try:
        sock.connect((mac_addr, port))
        print(f"[+] Connected to {mac_addr}")
        return sock
    except Exception as e:
        print(f"[-] Connection error to {mac_addr}: {e}")
        return None

def exchange_data(sock, device_id):
    if not sock: return
    #msg = f"Data from Master -> {device_id}\n"
    msg = "PING"
    sock.send(msg.encode('utf-8'))
    response = sock.recv(1024).decode('utf-8').strip()
    print(f"<- Response from {device_id}: {response}")

def main():
    print("=== Piconet initialization ===")
    s1 = connect_rfcomm(ESP1_MAC)
    s2 = connect_rfcomm(ESP2_MAC)

    if s1 and s2:
        print("=== Both channels active. Starting exchange ===")
        for i in range(5):
            exchange_data(s1, "ESP32_1")
            exchange_data(s2, "ESP32_2")
            time.sleep(1)

        s1.close()
        s2.close()
        print("=== Connections closed ===")
    else:
        print("[-] Failed to establish all connections")

if __name__ == "__main__":
    main()

```

2.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Задание согласно варианту.
4. Листинги результатов работы:
 - а) схема топологии созданной пикосети с указанием ролей и MAC-адресов;
 - б) фрагмент кода прошивки ESP32 с указанием места вставки ФИО;
 - в) листинг Python-скрипта с комментариями;

- d) вывод терминала с результатами подключения и подтверждением наличия ФИО в ответе устройства;
 - e) фрагменты лога `btmon`, подтверждающие установление L2CAP- и RFCOMM-каналов;
5. Ответы на контрольные вопросы.
 6. Краткие выводы по работе.

Примечание. При оформлении отчёта допускается сокращать листинги до ключевых фрагментов. Вывод `btmon` рекомендуется прилагать в виде вырезок, иллюстрирующих этапы установки соединения и передачи данных.

Контрольные вопросы

1. Что такое пикосеть и каковы её основные ограничения по количеству активных устройств?
2. Какую роль выполняет мастер в организации пикосети? Чем его функции отличаются от функций слейва?
3. Объясните принцип работы технологии FHSS. Почему скорость скачков в Bluetooth составляет именно 1600 гоп/с?
4. Как адрес мастера и его внутренние часы влияют на генерацию частотной последовательности?
5. Какую задачу решает протокол RFCOMM в стеке Bluetooth и как он взаимодействует с L2CAP?
6. Почему при программировании сокетов в Python используется константа `BTPROTO_RFCOMM`?
7. Каким образом обеспечивается изоляция логических каналов в рамках одного физического соединения?
8. Какие этапы установки RFCOMM-соединения можно наблюдать в логе `btmon`?
9. Какая группа пользователей в Linux необходима для доступа к последовательным портам при прошивке ESP32?